Propuesta recibida en marzo 6 2024 y aceptada para publicación en marzo 29 2024.

Trabajo Práctico 1: Análisis de esquemas de interpolación y solución de sistemas no lineales

Metodos Numéricos y Optimización

Ana Paula Tissera Francesca Giavarini

Abstract

La interpolación es un método de estimación a partir del cual se construyen funciones dado un conjunto de datos discretos conocidos o DataFrame para aproximar aquellos que no. En otras palabras, hallar alguna aproximación de la función que podría estar generando los datos. En este informe se analizará el rendimiento de diversos esquemas de interpolación al igual que la reconstrucción trayectorias. En primera instancia, se encuentra que el método de *Splines Cúbicos* minimiza el error con respecto de la aproximación de funciones de forma lineal o polinomial. Además, proponemos aprovechar el teorema de *Chebyshev* para mejorar el funcionamiento de los modelos polinomiales.

Luego, teniendo en cuenta estas conclusiones, utilizamos este método para reconstruir las trayectorias dadas y escribimos un algoritmo de $Newton - Raphson$ para hallar soluciones de sistemas no lineales.

Keywords

Interpolación, (Lagrange), Splines Cúbicos, Chebyshev,

1. Introducción

En la primera parte de este trabajo, se estudiará el rendimiento y cómo se ven afectados los distintos esquemas de interpolación de funciones conocidas al utilizar diversas cantidades de puntos de forma uniformemememente espaciada y no uniformemente espaciada. El método de interpolación utiliza puntos ya conocidos para predecir el valor de otros cercanos. Cada esquema es diferente en cómo relaciona los datos existentes con el resultado. Es por esto que, variar la cantidad y distancia entre los puntos ya conocidos afectará a cada polinomio interpolante de forma diferente en términos precisión y eficacia. Este trabajo tiene la intención de proporcionar una evaluación detallada de estos efectos.

Para la segunda parte, se modelizará un problema que busca simular la trayectoria de un vehículo a través del tiempo conociendo solamente algunas de sus posiciones en coordenadas $(x1, x2)$. Es así que nos encontramos en un entorno no lineal, con dos funciones a interpolar, una por cada coordenada, para lo que se utilizará el método de Splines Cúbicos. Seguidamente, se comparará la aproximación interpolada contra más mediciones de la curva real de trayectoria, teniendo en cuenta el error generado con los polinomios cúbicos.

Luego, se inserta un segundo vehículo con el fin de aproximar el punto de intersección entre ambos. Análogamente al vehículo 1, se aproxima su trayectoria y posteriormente, se utiliza el método de búsqueda de raíces de Newton para buscar dicha intersección.

2. Métodos empleados

2.1. *Método de interpolación a trozos (Splines) lineal*

La interpolación lineal es una manera rápida y simple de aproximar el valor de una funcion entre puntos dados. La estrategia es usar una línea recta para conectar los datos conocidos en el intervalo $[x0, x1]$. Cada recta se construye de la manera:

$$y = y0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0) \tag{1}$$

Se encuentra implementado en scipy como `scipy.interpolate.interp1d`.

En R^3 sigue la misma lógica pero con una variable adicional y se encuentra implementado en scipy como `scipy.interpolate.RectBivariateSpline` con kx=1, ky=1

2.2. *Método de Lagrange*

El método de Lagrange aproxima la función utilizando un único polinomio tal que para todos

los puntos dados $P(x_k) = f(x_k)$.

$$P(x) = \sum_{k=0}^{n} f(x_k) L_{n,k}(x) \tag{2}$$

$L_n, k(x)$ son las bases de Lgrange definidas como:

$$L_{n,k} = \prod_{i=0}^{n} \frac{x - x_i}{x_k - x_i} \tag{3}$$

Siendo x_k y x_i puntos del DataFrame.

Se encuentra implementado en scipy como `scipy.interpolate.lagrange`.

2.3. *Método de Splines Cúbicos para funciones de una variable*

Para este método, dividimos el intervalo $[a, b]$ en subintervalos $[x_j, x_{j+1}]$ y luego definimos una función para cada uno de la siguiente manera:

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3 \quad \text{con } x_j \leq x \leq x_{j+1} \tag{4}$$

Entonces, para cada subintervalo $[x_j, x_{j+1}]$, debemos encontrar los coeficientes a_j, b_j, c_j, d_j, mediante un sistema de ecuaciones lineal. Se encuentra implementado en scipy como `scipy.interpolate.CubicSpline`.

2.4. *Método de Splines Cúbicos para funciones de dos variables*

El método de splines cúbicos en dos variables sigue la misma lógica pero se utiliza para construir una superficie suave a través de un conjunto de puntos en tres dimensiones. Se basa en la idea de dividir el dominio en subdominios rectangulares definidos por los puntos de la malla y construir un polinomio cúbico en cada subdominio.

Los coeficientes $(a_{ij}, ..., j_{ij})$ se determinan para cada subintervalo resolviendo un sistema de ecuaciones lineales que obtiene al imponer que el spline cúbico coincida con los valores de la función en los puntos de la malla y que las primeras y segundas derivadas parciales sean continuas en los puntos interiores de la malla. Este proceso produce una superficie suave que pasa por todos los puntos de la malla y tiene una forma "flexible", lo que la hace útil para modelar formas complejas en tres dimensiones.

Se encuentra implementado en scipy como `scipy.interpolate.RectBivariateSpline` con kx=3, ky=3

2.5. *Método de Splines Quínticos para funciones de dos variables*

Los splines quinticos son una extensión de los splines cúbicos y, al igual que estos, son funciones polinómicas, pero de grado cinco en lugar de tres. O sea, se construye un polinomio de quinto grado para cada par de puntos de datos adyacentes, de manera que el spline completo es suave y diferenciable hasta la cuarta derivada en todos los puntos de datos.

Se encuentra implementado en scipy como `scipy.interpolate.RectBivariateSpline` con kx=5, ky=5

2.6. *Método de Newton-Raphson para sistemas de ecuaciones no lineales con dos variables*

El método de Newton-Raphson es una técnica iterativa para encontrar las raíces de un sistema de ecuaciones no lineales. Busca los puntos donde todas las ecuaciones se igualan a cero.

La idea básica del método es la siguiente: comienza con una estimación inicial P_0 y utiliza la derivada de las ecuaciones del sistema para guiar la convergencia hacia las raíces. En cada iteración, se calcula una corrección al punto actual que se suma a este punto para obtener el próximo, utilizando la fórmula:

$$P_{n+1} = P_n - J^{-1}(P_n) \cdot f'(P_n) \tag{5}$$

Este método es altamente efectivo pero puede requerir muchas iteraciones y una buena estimación inicial para converger correctamente. Si la aproximación x_k es suficientemente cercana a la solución exacta $x*$, el error se reduce cuadráticamente (p=2) en cada iteración del método de la siguiente manera:
La relación entre el error en la (k+1)-ésima y en la k-ésima iteración se puede expresar como:

$$\frac{\|\mathbf{x}_{k+1} - \mathbf{x}^*\|}{\|\mathbf{x}_k - \mathbf{x}^*\|^p} \approx C \tag{6}$$

2.7. *Teorema de puntos de Chebyshev*

Por último, todos los métodos de interpolación serán probados utilizando los puntos de Chebyshev además de los equiespaciados. Los nodos de Chebyshev en un intervalo $([a, b)]$ se pueden calcular utilizando la fórmula:

$$x_i = \tfrac{1}{2}(a + b) + \tfrac{1}{2}(b - a) \cdot \cos\left(\tfrac{2i-1}{2n} \cdot \pi\right) \quad \text{para } i = 1, \ldots, n \tag{7}$$

3. Desarrollo y Resultados

3.1. *Efecto del número y posición de los puntos de interpolación*

La interpolación de funciones en una variable se usa para estimar valores entre puntos conocidos a lo largo de una línea o curva, mientras que cuando hablamos de una función en varias variables se emplea para estimar valores en una superficie o en un espacio de múltiples dimensiones. Para evaluar cómo funcionan distintos métodos de interpolación, vamos a utilizar dos funciones: una que depende de una sola variable y otra que depende de dos.

3.1.1. Interpolación de una función conocida en una variable

Definimos

$$f_a(x) = 0{,}3^{||x||} \cdot \sin(4x) - \tanh(2x) + 2 \qquad (8)$$

como la primer función a interpolar. Para ello, utilizamos los métodos de interpolación lineal, Lagrange y Splines Cúbicos.

Vamos a comenzar por el método más simple, el lineal. La interpolación lineal asume que la función es aproximadamente lineal entre cada par de puntos. Al estar hablando de una función que tiene curvatura (o sea, su derivada segunda es distinta de cero), la interpolación lineal tiene un error más significativo mientras más separados estén los puntos, y, por ende, más grande sean los intervalos a interpolar. Este error tiende a ser mayor cerca del medio del intervalo entre los puntos, donde la función se desvía más de una línea recta.

Cuando se utilizan más puntos, la interpolación lineal puede se vuelve más precisa porque cada segmento lineal es más corto y por lo tanto es más probable que la función sea más parecida a una lineal en ese segmento. Sin embargo, hay que hacerlo moderadamente porque mientrás mayor es cantidad de puntos, más se incrementa el costo computacional.
Pero, ¿qué tipos de puntos nos conviene elegir?

Cuando los puntos están equiespaciados, la interpolación lineal captura la forma general de la función, pero tiende a perder detalles finos, especialmente en las regiones donde la función cambia rápidamente.

En contraste, si los puntos de interpolación están más juntos donde la función cambia rápidamente (con los nodos de Chebyshev), la interpolación lineal será más precisa en esos sectores.

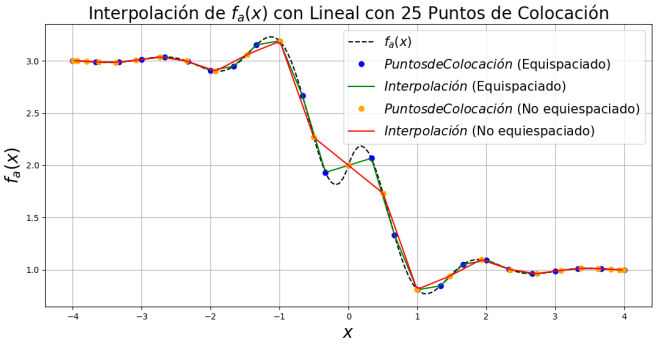

Figura 1: función interpolada con ambos tipos de puntos comparada con la original

Sin embargo, se pierde mucha información de la curvatura en el medio del intervalo, como se puede ver en la figura 1, lo que genera, como se ve en la figura 2, más error que los equiespaciados a nivel general para determinadas cantidades de puntos.

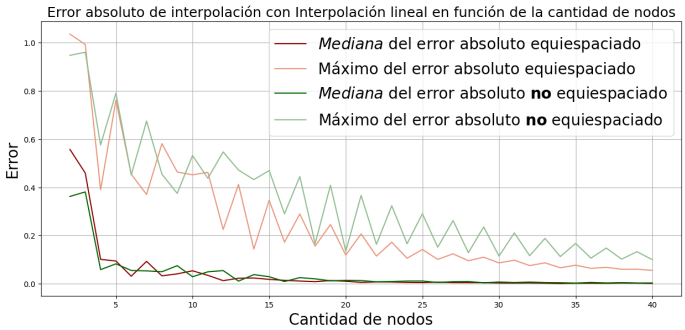

Figura 2

Esto quiere decir que no hay una elección única y definitiva sobre los tipos de puntos que conviene elegir cuando interpolamos linealmente, sino que depende de la cantidad de nodos sobre los cuales vamos a realizar la interpolación y de la distribución de la oscilación de nuestra función.

Como segundo método empleado, encontramos el de Lagrange.

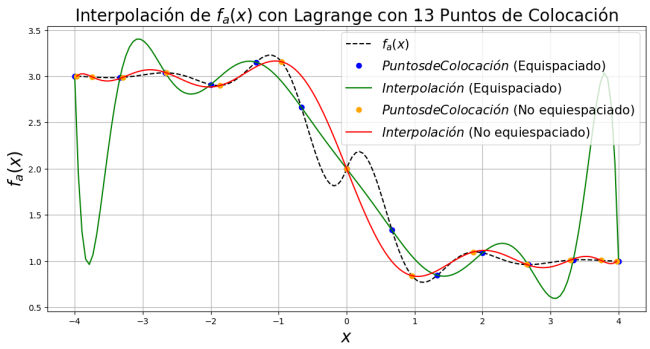

Figura 3

Los resultados obtenidos sugieren que, al utilizar este método, a medida que aumentamos la cantidad de puntos equiespaciados a evaluar, o sea el grado de nuestro polinomio interpolante, nos encontramos frente a un creciente fenómeno de Runge: el costo de interpolar con un único polinomio.

En este, a medida que aumenta el grado del polinomio interpolante, en lugar de obtener una mejor aproximación, este oscila de manera salvaje, especialmente cerca de los extremos del intervalo de interpolación, donde la derivada segunda es más alta. Esta oscilación se debe a que el polinomio de interpolación está tratando de ajustarse a la curvatura de la función, que es controlada por esta segunda derivada. Cuando intentamos ajustar un polinomio de alto grado a un conjunto grande de puntos, el polinomio tenderá a oscilar entre ellos para poder pasar exactamente por cada uno de ellos. Esto lleva, inevitablemente, a errores de interpolación muy grandes (ver figura 4).

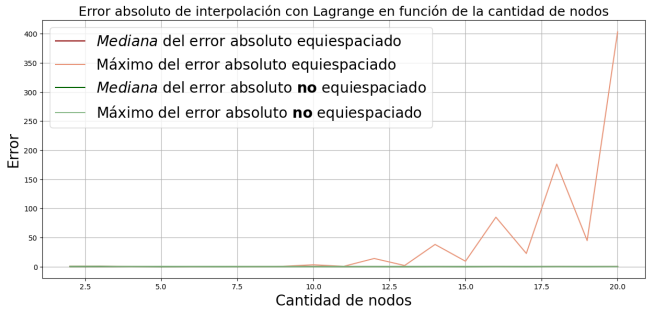

Figura 4

Sin embargo, encontramos que los nodos de Chebyshev pueden ayudar a mitigar este fenómeno de Runge. Estos nodos están más juntos cerca de los extremos del intervalo de interpolación, o sea, con Chebyshev se colocan más puntos cerca de las regiones donde la función puede tener cambios rápidos. Esto resulta en una interpolación más precisa, porque el polinomio de interpolación cambia más lentamente cerca de los

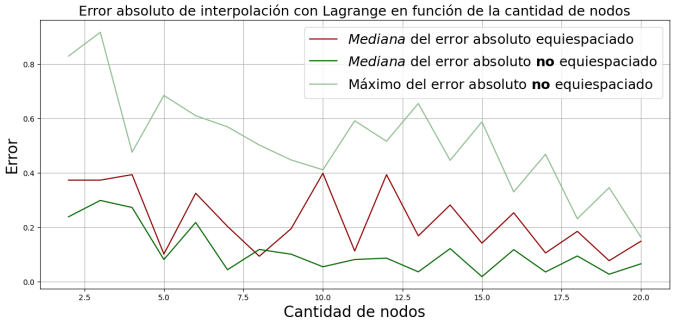

Figura 5: error por cantidad de nodos con zoom

extremos, lo que ayuda a reducir la oscilación y, por ende, el error. De igual manera, como se evidencia en la figura 5, incluso utilizando estos nodos, una cantidad muy grande de los mismos también puede resultar en una disparada del error interpolante.

El último método que utilizamos para estudiar el efecto del número y posición de los puntos de interpolación es Splines Cúbicos.

Como sabemos, este utiliza polinomios cúbicos en cada intervalo entre puntos de datos, y los elige de manera que la función interpolada sea suave en todo el rango de datos.

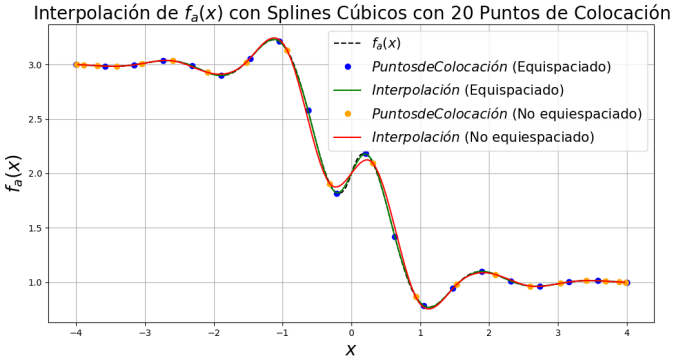

Figura 6

De esta manera, cuando los puntos están equiespaciados, los Splines Cúbicos capturan la forma general de la función, y, aunque pueda llegar a perder detalles finos, en general proporciona una buena aproximación. Es así que, al aumentar el número de nodos, es decir, al hacer que los puntos estén más juntos, se puede mejorar la precisión de la interpolación. Esto es porque se interpola en intervalos regulares cada vez más pequeños, entonces la aproximación tiene la posibilidad de acercarse suavemente, adaptándose mejor a las características locales de la función, lo que reduce notoriamente el error comparativamente a los métodos previamente analizados.

Al tratarse de los puntos no equiespaciados, como en el caso de los nodos de Chebyshev, la hipótesis era que Splines Cúbicos podría llegar a ser aún más preciso dada una función con más curvatura cerca de los extremos que en el medio. Sin embargo, como podemos visualizar en la figura 7, dependiendo de la cantidad de nodos a utilizar para construir nuestra $f(x)$ interpolante, es el espacio que conviene tomar entre ellos. Pero en general vamos a preferir una mayor cantidad de puntos distanciados regularmente sobre los que interpolar, para así evitar tener que interpolar sobre intervalos demasiado grandes, ya que así se pierde menos información general de lo que queremos aproximar.

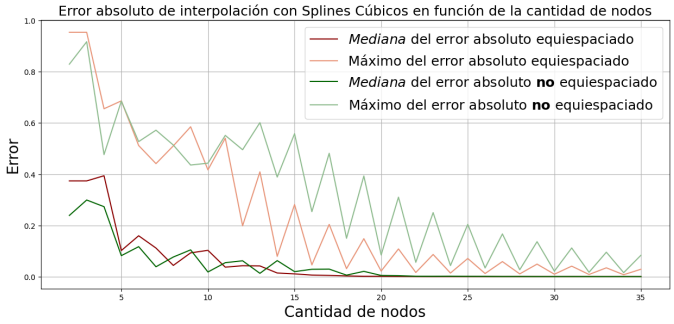

Figura 7

3.1.2. Interpolación de una función conocida en dos variables

Definimos la siguiente función a estudiar:

$$
\begin{aligned}
&f_b(x_1, x_2) = \\
&0{,}75 \cdot \exp\left(-\frac{(10x_1-2)^2}{4} - \frac{(9x_2-2)^2}{4}\right) \\
&+ 0{,}65 \cdot \exp\left(-\frac{(9x_1+1)^2}{9} - \frac{(10x_2+1)^2}{2}\right) \\
&+ 0{,}55 \cdot \exp\left(-\frac{(9x_1-6)^2}{4} - \frac{(9x_2-3)^2}{4}\right) \\
&- 0{,}01 \cdot \exp\left(-\frac{(9x_1-7)^2}{4} - \frac{(9x_2-3)^2}{4}\right)
\end{aligned}
\tag{9}
$$

con $x_1, x_2 \in [-1, 1]$.

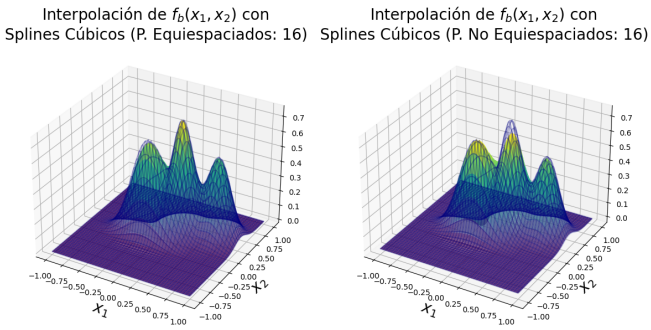

Figura 8

Para analizar este caso, decidimos focalizarnos en la interpolación a trozos o *Splines*, variando su grado para estudiar el desempeño y error de los mismos en cada caso. Primero decidimos relizar un estudio con Splines de grado 1, o sea lineales. Seguidamente decidimos aumentar el grado a 3 y posteriormente a 5. Los resultados se condicen con los del estudio de la interpolación de una función en una variable: al incrementar la cantidad de nodos, el error asociado a los Splines se ve notablemente reducido.

Sin embargo, los grados de estos polinomios tienen indudablemente un efecto en la precisón de nuestra aproximación. Si comparamos los errores generados por Splines Lineales y Cúbicos, que se encuentran representados en la figura 8 y 9 respectivamente, encontramos una diferencia significativa, no sólo en el valor máximo que alcanzan tanto la mediana como el máximo del error por cada cantidad de nodos utlizados, sino en la rápidez con la que el error se va acercando a 0. Mientras que con Splines de grado 3, a partir de los 12 nodos equiespaciados el error máximo ya se encuentra por debajo del 0.1, a los Splines de grado 1 le toma más de 18 nodos para poder pasar ese umbral. Esto quiere decir que para llegar al mismo grado de precisión, el primero requiere menos puntos, lo que equivale a menos información, que haciéndolo por partición lineal. Asimismo, es alrededor del punto 25 que el error se vuelve prácticamente despreciable para Splines Cúbicos, pero para interpolación Lineal podemos notar que incluso con 50 nodos el error sigue siendo perceptible. Esto se condice con lo que habíamos visto anteriormente: Splines Cúbicos ajusta la función de una forma mucho más suave que cómo lo hacen las rectas, por lo que logra un error relativamente mucho más pequeño.

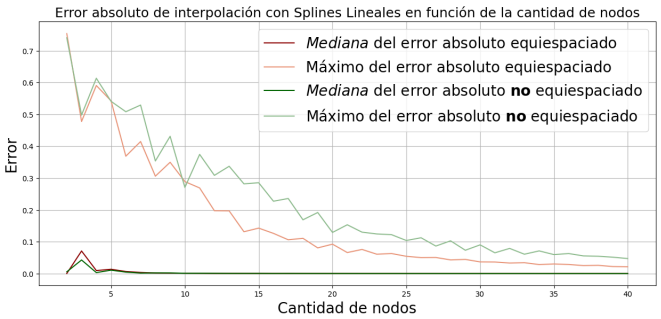

Figura 9

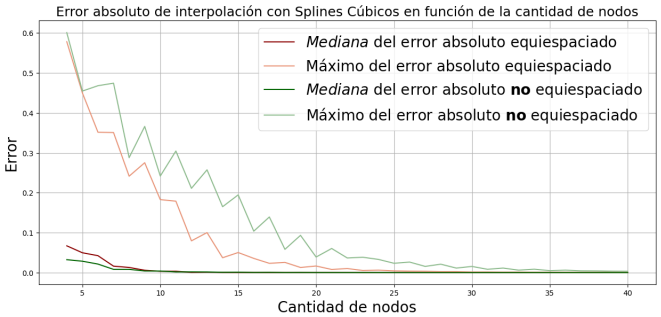

Figura 10

El siguiente método empleado son Splines Quínticos. En vista de lo anterior, es de esperar que aumentar el grado de los polinomios utilizados para interpolar pueda llegar a aumentar la precisión al aproximar nuestra función, al posiblemente hacerlo de manera más suave. De he-

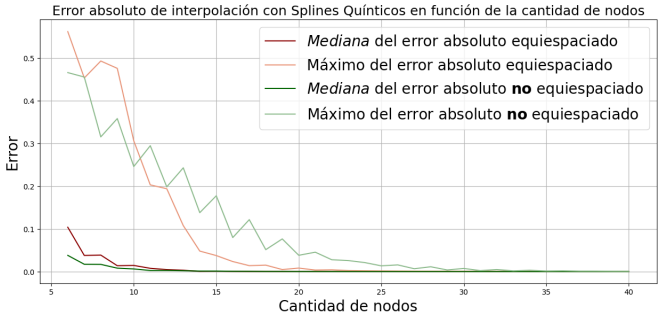

Figura 11

cho, si miramos la figura 10, podemos notar que esto efectivamente se condice pero en determinadas cantiadades de nodos. A diferencia de la comparación anterior, donde podíamos establecer un método que en términos de error era siempre superior, en este caso, comparandolos con los nodos equiespaciados, si bien en general Splines Quínticos proporciona una mejor aproximación, hay determinadas cantidades en donde Splines Cúbicos cuenta con menos error. Como por ejemplo con la cantidad de 10 nodos Splines de grado 3 tiene un error máximo de aproximadamente 0.1827 y mediano de 0.00316, mientras que con grado 5 tiene 0.0143996 de error máximo y 0.30615 de mediano, lo cual es superior. Igualmente, comc estaba mencionando, generalizando, el error (Splines Quínticos es levemente inferior (se p de ver comparando en otros puntos), y se ace a 0 de una manera muy similar en términos cantidad de nodos necesarios.

En cuánto a la utilización de nodos Chebyshev, para Splines de primer y tercer g do, a simple vista, estos no parecen mejorar error. Estudiando los errores podemos ver que la diferencia entre usar unos u otros no es muy significativa y que, si bien el error mediano por cantidad de nodos es relativamente apenas menor, el máximo de los nodos de Chebyshev es en general mayor.

Es así que, la interpretación de los resultados tiene correlación con lo que se había expuesto analizando la interpolación por partición de la función $fa(x)$, en donde se terminaba por decir que utilizar puntos equidistantes llega a resultar en una estabilidad mayor en la estimación de la función, ya que evita intervalos de gran tamaño innecesariamente.

En cuánto a los Splines Quínticos, vemos que, para cantidades más chicas de puntos, los nodos elegidos con el Teorema de Chevysheb sí mejoran el error. Esto se debe a que, al aumentar el grado del spline (en este caso, de cúbico a quíntico), se incrementa la flexibilidad del spline y esto puede llevar a una mayor susceptibilidad al fenómeno de Runge previamente explicado y, como habíamos desarrollado, la elección estratégica de estos nodos podía ayudar a mitigar sus efectos. Sin embargo, a partir de 11 datos esto cambia, lo que nos lleva a decir, otra vez, que no hay una elección de puntos que sea mejor en su totalidad, sino que en general va a depender de la cantidad de información con la que contemos, la distribución de la oscilación de nuestra función y la complejidad algorítmica que estemos dispuestos a sumarle.

3.2. *Construcción de trayectoria*

3.2.1. Reconstrucción de la trayectoria de un vehículo a partir de puntos dados y comparación contra más mediciones de su Ground-Truth

Para recosntruir la trayectoria del vehículo sólo contamos con 10 puntos por los que pasó registrados a lo largo del tiempo en dos coordenadas $(x1, x2)$ $(mediciones)$ y una mayor cantidad los mismos que representan la trayectoria real $(ground\ truth)$ contra los que comparar nuestra interpolación. Lo que se busca estimar los segundos a partir de los primeros y analizar el error.

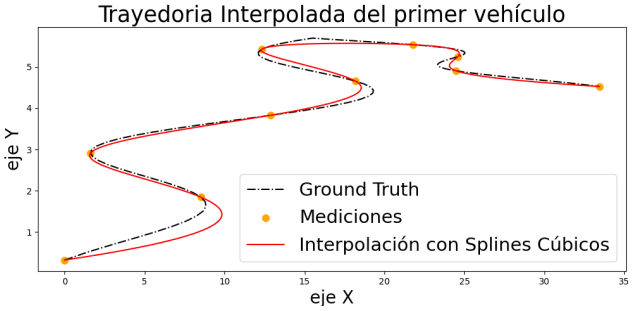

Figura 12

Dada su conveniente precisión en la estimación de la función real y su cálculo no es excesivamente complejo, decidimos realizar dicha aproximación con Splines Cúbicos. La interpolación fue realizada para cada coordenada de manera tal que $\sigma_1(t) = (interp\ de\ los\ puntos\ en\ x_1, interp\ de\ los\ puntos\ en\ x_2)$.

Esto nos implicó encontrarnos con el siguiente error (calculado a partir de la distancia entre los puntos con la norma euclideana):

Entre los datos a destacar nos contontramos con la mediana, que vale aproximadamente 0,58449 y la diferencia más grande entre la interpolación y el *ground truth*, o sea el error máximo, que fue de 2,36468. Esto deja en evidencia que, al contar con pocos puntos, la aproximación tiene errores que superan en magnitud a los decimales,

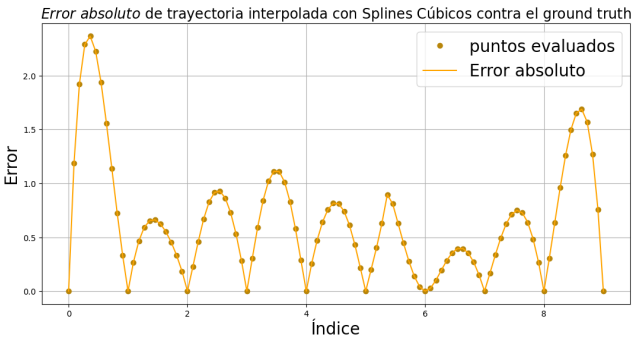

Figura 13

pero juzgando por la mediana, en general estos se encuentran dentro de ese margen.

3.2.2. Resolución de un sistema no lineal para encontrar la intersección entre las trayectorias de 2 vehículos

Siguiendo la lógica anteriormente aplicada, para aproximar la trayectoria del segundo vehículo utilizamos Splines Cúbicos obteniendo $\sigma_2(t')$

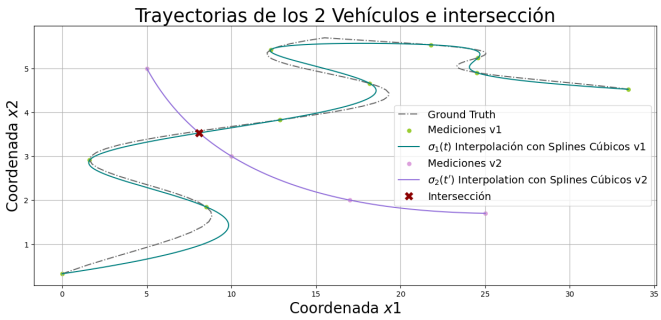

Figura 14

Para encontrar analíticamente la intersección entre ambas trayetorias ($\sigma_1(t) = \sigma_2(t')$) planteamos el sistema no lineal $\sigma_1(t) - \sigma_2(t') = 0$, que resolvimos a partir de la implementación en dos variables del método de Newton-Raphson propiamente construido.

La idea era plantear una función[1] que representara el proceso iterativo descripto por la fórmula:

$$P_{n+1} = P_n - J^{-1}(P_n) \cdot f'(P_n)$$

explicada en el apartado de "Métodos empleados".

El método convergió en 8 iteraciones, encontrando a los tiempos aproximadamente $t = 2{,}66116$ y $t' = 0{,}67315$. Al evaluar en las interpolaciones obtenidas para cada trayectoria se encuentra como coordenadas de la intersección a: (8,09706957, 3,529095804)

Estas coordenadas difieren según evaluadas con una u otra interpolación σ en su tiempo correspondiente en el décimo dígito significativo en x_1 y en el onceavo en x_2, por lo que prácticamente representan el mismo punto con una tolerancia asignada de 1e-6.

Conclusiones

En este informe se han investigado diversos métodos de interpolación en una amplia gama de contextos aplicados, abarcando tanto funciones univariables como multivariables. Se han analizado escenarios en los cuales se dispone de información completa sobre las funciones originales a interpolar, así como aquellos en los que esta información es limitada o inexistente.

Al examinar detalladamente cada método de interpolación, se ha llegado a la conclusión de que cada uno posee sus propias ventajas así como limitaciones. Por ejemplo, los métodos de *Lagrange* y de interpolación *lineal* destacan por su simplicidad en la implementación y su reducido costo amortizado, que es lineal en función de la cantidad de nodos n. Sin embargo, se ha observado que generan un error significativamente mayor en determinados casos en comparación con algoritmos más complejos como los *Splines Cubicos*, los cuales reducen el error de manera notable a expensas de un mayor costo algorítmico, dado que requieren cálculos adicionales para ajustar los polinomios de tercer grado a intervalos entre puntos de datos, lo que puede llevar más tiempo según el tamaño del conjunto de datos en comparación con los métodos de interpolación más simples.

También, se ha observado que los algoritmos de interpolación con un único polinomio, en nuestro estudio *Lagrange*, tienden a producir oscilaciones en los resultados cuando se quiere interpolar utilizando muchos nodos, dado que intentan pasar por todos los puntos, lo cual termina por incrementar el error en los puntos a estimar. Es así que probamos que esto mejora sustancialmente con la utilización de los nodos de Chebyshev en lugar de puntos equiespaciados. Sin embargo, al trabajar a trozos con varios polinomios cuando se trata de Splines, estos nodos no reducen el error y, algunas veces, hasta incluso lo incrementan.

Sabemos que aumentar el grado de los polinomios de Splines le suma complejidad, y probamos que, por ejemplo, utilizando *Splines Quinticos*, la diferencia en el error no resulta lo suficientemente significativa como para justificar esta mayor complejidad algorítmica asociada. De esta manera, continuamos el resto del análisis con el

método que consideramos balanceaba de mejor manera costos y precisión: *Splines Cubicos*.

Por último, estudiamos la resolución de un sistema no lineal a partir del algoritmo de Newton-Raphson y observamos que converge rapidamente y de forma precisa.

De este modo, este informe comparó diferentes esquemas de interpolación, a fin de determinar cuáles son los más convenientes para la estimación de funciones complejas en términos de los datos disponibles.

Apéndice

1. Explicación de la función que construimos del método de Newton-Raphson para un sistema de dos ecuaciones no lineales:

En términos de ecuaciones, para representar el proceso iterativo del método hay que resolver el sistema lineal que se puede escribir como:

$$\begin{bmatrix} \frac{\partial f_1}{\partial x}(P_n) & \frac{\partial f_1}{\partial y}(P_n) \\ \frac{\partial f_2}{\partial x}(P_n) & \frac{\partial f_2}{\partial y}(P_n) \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = - \begin{bmatrix} f_1(P_n) \\ f_2(P_n) \end{bmatrix}$$

Donde f_1 y f_2 son las dos ecuaciones del sistema, P_n es el punto actual, y Δx y Δy son las correcciones a las variables x y y que se necesitan para acercarse a las raíces del sistema de ecuaciones. Se desarrolla de la siguiente manera:

1. Se inicializa el punto P con los valores iniciales P_0 y P_1. Este punto representa la estimación actual de las raíces del sistema de ecuaciones.
2. Se entra en un bucle que se ejecuta un número máximo de veces para evitar que el algoritmo entre en un bucle infinito en caso de que no converja o que lo haga muy lentamente (max_iter). En cada iteración:
   a. Se evalúa la función f en el punto actual P. Esto da un vector f_{val} de dos elementos, que son los valores de las dos ecuaciones en el punto P. Si su norma de es menor que un cierto umbral (tol), se detiene el bucle indicando que P es una raíz del sistema de ecuaciones a una tolerancia de tol.
   b. Se calcula la matriz Jacobiana J de f en el punto P. A esta la calculamos utilizando diferencias finitas con un paso de tol.
   c. Se resuelve el sistema lineal $J \cdot \delta_P = -f_{\text{val}}$ para encontrar δ_P. Esto da la corrección al punto actual P que se necesita para acercarse a las raíces del sistema de ecuaciones.
   d. Se actualiza el punto P sumando δ_P.
3. Al finalizar se devuelve el punto P.

4. Referencias

- **Burden, A., Burden, R., Faires, D.** (2014). *Numerical Analysis*. Cengage.
- **Ingeniero Castro P, C.** (s.f.). Facultad de Ingeniería de Minas, Geología y Civil. Obtenido de https://cristiancastrop.wordpress.com/wp-content/uploads/2010/09/catedra-metodos-numericos-2013-unsch-06.pdf
- **Rodríguez, D. M.** (s.f.). Obtenido de https://biblus.us.es/bibing/proyectos/abreproy/3989/fichero/MEMORIA%252F13_ANEXO7.pdf
- **Sachs, J. M.** (30 de Septiembre de 2012). Embedded. Obtenido de https://www.embeddedrelated.com/showarticle/152.php
- **The SciPy community.** (s.f.). SciPy v1.12.0 Manual. Obtenido de https://docs.scipy.org/doc/scipy/reference/interpolate.html